Data Analysis with Python 2024–2025

Parte 6: Machine Learning Types

Soluções dos Exercícios

Tradução e Adaptação: Universidade de Helsinque (MOOC.fi)

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **Universidade de Helsinque (MOOC.fi)**. As soluções apresentadas a seguir foram elaboradas para fins didáticos, seguindo os enunciados dos exercícios propostos no curso original.

Conteúdo

1 Soluções - Capítulo 6

Exercício 6.1 – Classificação de Blobs

Solução proposta

```
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def blob_classification(X, y):
    # Divide os dados de maneira deterministica (random_state=0)
       com tamanho de treino 75%
    X_train, X_test, y_train, y_test = train_test_split(X, y,
       train_size=0.75, random_state=0)

    # Treina o modelo usando Naive Bayes Gaussiano
    model = GaussianNB()
    model.fit(X_train, y_train)

    # Efetua as previsoes e retorna a acuracia
    predictions = model.predict(X_test)
    return accuracy_score(y_test, predictions)
```

Exercício 6.2 – Classificação de Plantas

Solução proposta

```
from sklearn.datasets import load_iris
from sklearn.naive_bayes import GaussianNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def plant_classification():
    # Carrega o conjunto local e divide com base em 80% do traning
       set
    dataset = load_iris()
    X = dataset.data
    y = dataset.target

    X_train, X_test, y_train, y_test = train_test_split(X, y,
       train_size=0.80, random_state=0)

    # Aplica o modelo Gaussiano
    model = GaussianNB()
    model.fit(X_train, y_train)

    # Retorna o score final
    return accuracy_score(y_test, model.predict(X_test))
```

Exercício 6.3 – Classificação de Palavras

Solução proposta

```
import numpy as np
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import cross_val_score, KFold

ALPHABET = "abcdefghijklmnopqrstuvwxyzäö-"

def get_features(a):
    # a: array 1D com palavras
    features = np.zeros((len(a), len(ALPHABET)))
    for i, word in enumerate(a):
        for char in word:
            if char in ALPHABET:
                features[i, ALPHABET.index(char)] += 1
    return features

def contains_valid_chars(s):
    # Verifica validade exclusiva pelas letras do alfabeto
       permitido
    return all(c in ALPHABET for c in s)

def get_features_and_labels():
    # Simulando as cargas de listas locais:
    # finnish = load_finnish()
    # english = load_english()

    # (Para uso de solucao online, consideraremos lists importadas)

    # Processa finlandes
    finnish_filtered = []
    for w in finnish:
        w_lower = w.lower()
        if contains_valid_chars(w_lower):
            finnish_filtered.append(w_lower)

    # Processa ingles (Removendo nomes proprios / Letra maiuscula
       primeiro)
    english_filtered = []
    for w in english:
        if not w[0].isupper():
            w_lower = w.lower()
            if contains_valid_chars(w_lower):
                english_filtered.append(w_lower)

    X = get_features(np.array(finnish_filtered + english_filtered))
    y = np.array([0]*len(finnish_filtered) + [1]*len(
       english_filtered))
    return X, y

def word_classification():
    X, y = get_features_and_labels()
    model = MultinomialNB()

    # Usa cross-validation kfold para prever de forma fragmentada (
       em 5 iteracoes)
    cv = KFold(n_splits=5, shuffle=True, random_state=0)
    scores = cross_val_score(model, X, y, cv=cv)
    return scores
```

Exercício 6.4 – Detecção de Spam

Solução proposta

```
import gzip
import numpy as np
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

def spam_detection(random_state=0, fraction=1.0):
    # Le os textos do ambiente src e captura as linhas referentes
       as fracoes processadas
    with gzip.open('src/ham.txt.gz', 'rt', encoding='utf-8') as f:
        ham_lines = f.readlines()
        ham = ham_lines[:int(fraction * len(ham_lines))]

    with gzip.open('src/spam.txt.gz', 'rt', encoding='utf-8') as f:
        spam_lines = f.readlines()
        spam = spam_lines[:int(fraction * len(spam_lines))]

    X = ham + spam
    y = np.array([0]*len(ham) + [1]*len(spam))

    # Inicializa a maquina de pipeline que converte letras
       repetidas por caracteristica
    vec = CountVectorizer()
    X_features = vec.fit_transform(X)

    # Divide dados
    X_train, X_test, y_train, y_test = train_test_split(X_features,
        y, train_size=0.75, random_state=random_state)

    # Classifica
    model = MultinomialNB()
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    # Resultados
    acc = accuracy_score(y_test, predictions)
    total_tested = len(y_test)
    misclassified = (y_test != predictions).sum()

    return acc, total_tested, misclassified
```